

Tutorial de Avaliação da Ferramenta

Recomendação de Refatorações via Debate Multiagente — guia para avaliar a ferramenta sobre projetos open source do GitHub

Este tutorial mostra, passo a passo, como usar a ferramenta para **avaliar empiricamente** a abordagem proposta no artigo, executando o *pipeline* sobre repositórios Python de código aberto e coletando os indicadores do **Plano de Validação Científica (Seção 4.3)**.

A metodologia, conforme o artigo, **descarta médias estatísticas** e se baseia exclusivamente em **frequências** e **medianas**, para evitar a distorção causada por *outliers*.

1. O que você vai medir

O artigo define a Pergunta Principal (PP) e três subperguntas, além de cinco indicadores. A ferramenta foi instrumentada para produzir todos eles automaticamente.

Pergunta / Indicador	Significado	Onde aparece na saída
Q1	Diversidade de oportunidades de refatoração (multiagente vs. agente único)	q1_distinct / baseline_q1_distinct
Q2	Amplitude de atributos de qualidade contemplados (0–3)	q2_attributes
Q3	Conflitos de design explícitos entre agentes	q3_conflicts, conflict_types
(iv)	Mediana do tempo de processamento por requisição	total_ms
(v)	Variação das métricas estáticas/dinâmicas antes/depois	maintainability_index, max_cc, ...

A suíte de avaliação (`scripts/evaluate.py`) consolida tudo isso em **medianas** e **frequências** prontas para o artigo.

2. Instalação e preparação

Pré-requisitos: `uv` (gerencia o Python 3.12 automaticamente). Opcionalmente Ollama (modelos locais, como no artigo) e Docker (SonarQube).

```
# 1. Instalar dependências (cria a venv Python 3.12)
uv sync --extra dev

# 2. Criar o arquivo de configuração
```

```
cp .env.example .env

# 3. (Opcional) validar que tudo roda, em modo heurístico
uv run refactoring-debate examples/sample_code.py
```

A ferramenta roda **sem Ollama, Docker ou chave de API** usando o provedor `heuristic`. Para os experimentos do artigo, recomenda-se um backend de LLM (próxima seção).

3. Escolha do backend de LLM (impacta a avaliação)

O backend é definido por `RD_LLM_PROVIDER` no `.env`. A escolha **afeta diretamente** o que você está medindo, então registre-a nos resultados.

Backend	Como ativar	Quando usar
<code>ollama</code>	<code>ollama serve + ollama pull llama3</code>	Reproduzir o artigo (Llama3 local)
<code>anthropic</code>	<code>uv sync --extra anthropic + chave</code>	Modelo hospedado mais forte (Claude)
<code>openai</code>	chave da OpenAI	Alternativa hospedada
<code>heuristic</code>	padrão sem LLM	<i>Smoke test</i> / piso determinístico

Importante para Q1 (diversidade): o ganho do multiagente sobre o agente único aparece com mais clareza com um LLM real, em que os especialistas divergem de fato. No modo `heuristic`, o *baseline* é um piso determinístico e o contraste de Q1 tende a ser menor — porém Q2 e Q3 já evidenciam o efeito da especialização e do debate.

Cada resultado registra o backend usado (campo `llm` em `summary.json` e `llm_model` na resposta da API), garantindo rastreabilidade entre execuções.

4. Passo 1 — Selecionar e clonar projetos open source

Critérios sugeridos para o corpus: projetos Python populares, de tamanho variado, com estilos de código diferentes. Use `git clone --depth 1` para baixar só o estado atual.

```
mkdir -p corpus && cd corpus
git clone --depth 1 https://github.com/psf/requests
git clone --depth 1 https://github.com/pallets/flask
git clone --depth 1 https://github.com/tiangolo/typer
cd ..
```

Boas práticas de amostragem (a suíte aplica filtros automaticamente):

- arquivos de teste, *migrations*, `.venv`, `build`, etc. são ignorados;
- arquivos muito grandes são pulados (`--max-loc`, padrão 400 linhas);

- limite o nº de arquivos por repositório com `--max-files` (padrão 60).

5. Passo 2 — Executar a avaliação em lote

O comando principal roda o debate multiagente sobre cada arquivo `.py` e agrega os indicadores. Use `--baseline` para incluir o agente único (necessário para Q1).

```
# Avaliar um repositório com baseline de agente único
uv run python scripts/evaluate.py corpus/requests --baseline

# Limitar a amostra e escolher a pasta de saída
uv run python scripts/evaluate.py corpus/flask \
  --baseline --max-files 40 --out results/flask

# Mais rodadas de crítica cruzada (com LLM)
uv run python scripts/evaluate.py corpus/typer --baseline --rounds 2
```

Segurança: por padrão a análise é **estática** (não executa o código). A flag `--dynamic` liga Scalene/cProfile/CodeCarbon, que **executam o código** — só use em código confiável e, idealmente, dentro de um contêiner/sandbox.

Saídas geradas em `results/<repo>/` (ou `evaluation_results/<repo>/`):

- `per_file.csv` — uma linha por arquivo, com todos os indicadores;
- `summary.json` — agregados (medianas e frequências) + metadados;
- `summary.md` — o mesmo resumo em formato legível.

6. Passo 3 — Interpretar as saídas

Exemplo de `summary.md` produzido pela suíte:

```
## Q1 – Diversity of refactoring opportunities
- Median distinct recommendations (multi-agent): 5.0
- Median distinct (single-agent baseline): 3.0

## Q2 – Quality attributes covered (0-3)
- Median attributes covered (multi-agent): 3.0
- Median attributes (single-agent baseline): 1.0
- Coverage frequency: {'1': 6, '2': 1, '3': 5}

## Q3 – Explicit design conflicts
- Files with >=1 conflict: 6 (50% of files)
- Median conflicts per file: 1.0
- Conflict type frequency: {'performance_vs_architecture': 9, ...}

## Processing time & metric baselines
- Median total time per file: 1126 ms
- Median maintainability index: 58.19
```

Colunas principais do `per_file.csv`:

Coluna	Indicador	Leitura
q1_distinct	Q1	nº de recomendações distintas (multiagente)
baseline_q1_distinct	Q1	idem para o agente único
q2_attributes	Q2	quantos dos 3 atributos foram cobertos
q3_conflicts	Q3	nº de conflitos explícitos
conflict_types	Q3	tipos (ex.: performance_vs_sustainability)
n_sustainability / n_architecture / n_performance	Q2	recomendações por dimensão
total_ms	(iv)	tempo total do <i>pipeline</i>
maintainability_index, max_cc, code_smells	(v)	métricas para antes/depois

7. Passo 4 — Baseline de agente único (Q1)

Para responder Q1 ("o multiagente aumenta a diversidade?"), a suíte executa, no mesmo arquivo e com as mesmas métricas, um **único agente genérico** que analisa tudo em uma só passagem, sem especialização nem debate (o "escopo de observação limitado" do artigo).

A comparação fica em:

- q1_distinct (multiagente) **vs.** baseline_q1_distinct (agente único);
- no summary, q1_median_distinct_recommendations vs. q1_baseline_median_distinct.

Espera-se que o sistema multiagente identifique **mais oportunidades distintas** e cubra **mais atributos** (Q2), enquanto o agente único, sem debate, **não produz conflitos** (Q3 = 0 por construção).

8. Passo 5 — Indicador (v): variação de métricas antes/depois

O indicador (v) mede como as métricas mudam **após** aplicar as refatorações recomendadas. Fluxo sugerido:

```
# 1. Snapshot ANTES (rápido, sem LLM)
uv run python scripts/evaluate.py corpus/requests \
  --metrics-only --out results/requests_before

# 2. Aplique as refatorações recomendadas em um clone do repositório
#    (manualmente, ou com auxílio de um LLM), gerando corpus/requests_refactored

# 3. Snapshot DEPOIS
uv run python scripts/evaluate.py corpus/requests_refactored \
  --metrics-only --out results/requests_after
```

```
# 4. Compare as medianas dos dois summary.json
```

O modo `--metrics-only` roda apenas a camada de ferramentas (Radon, Pylint, import-linter e — com `--dynamic` — Scalene/cProfile/CodeCarbon), produzindo as medianas de `maintainability_index`, `max_cc`, `code_smells` e energia. A diferença entre os snapshots ANTES e DEPOIS é o indicador (v).

9. Passo 6 — Consolidar entre vários repositórios

O artigo trabalha com **frequências e medianas** sobre o conjunto. Rode a suíte em cada repositório do corpus e consolide os `summary.json`. Cada `summary.json` já traz:

- **medianas:** `q1_median_*`, `q2_median_*`, `q3_median_*`, `median_total_ms`, `median_maintainability_index`, `median_max_cyclomatic_complexity`;
- **frequências:** `q2_attribute_coverage_frequency` (arquivos por nº de atributos), `q3_freq_files_with_conflict`, `q3_conflict_type_frequency`.

Para o artigo, reporte a mediana **por repositório** e a mediana **do conjunto** das medianas; e as frequências agregadas (ex.: "em 50% dos arquivos emergiu ao menos um conflito; o tipo mais frequente foi `performance_vs_architecture`").

Lembre-se: **não use médias**. Use medianas e distribuições de frequência, como define a Seção 4.3.

10. Reprodutibilidade e boas práticas

- **Fixe as versões:** registre o commit clonado (`git rev-parse HEAD`), a versão da ferramenta e o `RD_LLM_MODEL`. Com `git clone --depth 1`, anote a data.
- **Temperatura baixa:** `RD_LLM_TEMPERATURE=0.2` (padrão) reduz variação entre execuções com LLM. Para LLMs, rode 2–3 vezes e reporte a mediana, pois há não-determinismo.
- **Amostragem consistente:** use os mesmos `--max-files` e `--max-loc` em todos os repos.
- **Pesos do juiz:** `RD_WEIGHT_*` mudam a arbitragem dos *trade-offs* (Q3). Documente os pesos usados; varie-os para um estudo de sensibilidade, se desejar.
- **Segurança:** só use `--dynamic` em código confiável e isolado.

11. Mapa de referência — pergunta → saída

Item do artigo	Arquivo	Campo
Q1 (diversidade)	summary.json	<code>q1_median_distinct_recommendations</code> , <code>q1_baseline_median_distinct</code>
Q2 (atributos)	summary.json	<code>q2_median_attributes_covered</code> , <code>q2_attribute_coverage_frequency</code>

Q3 (conflitos)	summary.json	q3_freq_files_with_conflict, q3_conflict_type_frequency
(iv) tempo	summary.json	median_total_ms
(v) métricas	summary.json (before/after)	median_maintainability_index, median_max_cyclomatic_complexity, median_code_smells
Evidência por arquivo	per_file.csv	todas as colunas
Registro do debate	API /api/v1/analyze	debate.conflicts, debate.tradeoffs

12. Solução de problemas

Sintoma	Causa provável	Solução
llm: heuristic mesmo com Ollama	Ollama não está rodando / modelo não baixado	ollama serve e ollama pull llama3
anthropic cai para heurístico	Falta o extra do provedor	uv sync --extra anthropic + RD_LLM_API_KEY
sonarqube: unavailable	Servidor não configurado	docker compose up -d sonarqube e definir RD_SONARQUBE_URL/TOKEN
py-spy: unavailable (macOS)	py-spy exige privilégios	rodar com sudo, ou ignorar (degrada sozinho)
Avaliação lenta	Pylint roda por arquivo	reduza --max-files; mantenha estático (sem --dynamic)

Exemplo mínimo, reproduzível de ponta a ponta

```
uv sync --extra dev
cp .env.example .env
git clone --depth 1 https://github.com/pallets/typer corpus/typer
uv run python scripts/evaluate.py corpus/typer \
  --baseline --max-files 30 --out results/typer
cat results/typer/summary.md
```

Ao final, results/typer/summary.md traz, em frequências e medianas, as respostas a Q1, Q2 e Q3 e os tempos — o material bruto para a avaliação do artigo.